

ANSIBLE_TASK_02

What is Ansible?

Ansible is an **automation tool** used to:

- Configure servers
- Install software
- Deploy applications
- Manage infrastructure

 It works using **YAML playbooks** and **agentless architecture**.

Importance of Ansible

1 Agentless (Very Important)

- No agent/software needed on worker nodes
- Uses **SSH (Linux)** and **WinRM (Windows)**
- Easy to manage thousands of servers

 Less maintenance

 More secure

2 Simple & Human-Readable

- Written in **YAML**
- Easy to learn compared to scripts

Example:

- name: Install nginx

apt:

name: nginx

state: present

Even non-programmers can understand it 

3 Automation Saves Time

Instead of manually:

- Logging into 100 servers
- Running same commands

Ansible does it in **one command**.

 Faster deployments

 No human errors

4 Idempotent Nature

Means:

Running the same playbook multiple times will not break anything.

If software already exists → Ansible skips it.

This is very important for **production stability**.

5 Used with DevOps Tools

Ansible integrates with:

- Jenkins
- Docker
- Kubernetes
- AWS / Azure / GCP

That's why Ansible is widely used in **CI/CD pipelines**.

Ansible Architecture

Ansible Controller

|

SSH / WinRM

|

| | |
Worker Worker Worker
Node Node Node

What is Ansible Controller?

The **controller** is the main machine where:

- Ansible is installed
- Playbooks are written
- Inventory is stored
- Commands are executed

Controller contains:

- Playbooks (YAML)
- Inventory file
- Roles
- Variables
- SSH keys

 Example:

Your laptop or Jenkins server can be controller.

What are Worker Nodes?

Worker nodes are:

- Target servers
- EC2 instances
- Linux / Windows machines

They do **not** need Ansible installed.

They only need:

- SSH access

- Python (usually preinstalled)
-

Importance of Controller–Worker Communication

This is **one of the most important concepts in Ansible** 🙌

1 Uses Secure Communication (SSH)

Controller → Worker communication happens via:

- **SSH (port 22)** for Linux
- **WinRM** for Windows

Benefits:

- Encrypted
 - Secure
 - No agents required
-

2 Push-Based Model

Ansible uses **push model**:

- Controller pushes tasks to workers
- Workers just execute commands

Unlike Puppet/Chef (pull model).

-  Better control
 -  Immediate execution
-

3 Inventory-Based Management

Controller uses inventory file:

[web]

server1

server2

[db]

server3

This allows:

- Group-wise execution
 - Environment separation (dev, qa, prod)
-

Scalability

One controller can manage:

- 10 servers
- 100 servers
- 10,000 servers

Because communication is lightweight SSH.

Centralized Control

Only controller has:

- Logic
- Credentials
- Playbooks

Workers never store sensitive data.

 Better security model.

Flow of Communication (Easy)

1. Controller reads playbook
2. Controller checks inventory
3. Controller connects to worker via SSH
4. Executes module temporarily
5. Gets result
6. Disconnects

No permanent service running on worker 

Real-Time Example

You want to install nginx on 50 servers.

Without Ansible:

- Login 50 times
- Run same command

With Ansible:

ansible-playbook install-nginx.yml

Done in seconds 

Tasks :

1. Watch ansible-02 video and write down notes.
2. Install httpd using ansible playbook, use handlers,notifiers.
3. Write a ansible playbook to install apache tomcat.
4. Write a ansible playbook to provision one ec2 on aws.
5. Write a ansible playbook to copy one file from node-1 to node-2.
6. Write a ansible playbook to create different files with different names using single playbook.

Task 2:

2. Install httpd using ansible playbook, use handlers, notifiers.

Step 1:

—> first connect to your two workers servers with

Ssh password less authentication

—> generate ssh-keygen in ansible controller

—> copy pub-key and paste it inside the worker1(node1)and worker2(node2)—>.ssh—>authentication-keys

—> add paste the public key and save exit

→ create ssh key as a ubuntu user

```
ubuntu@172-31-24-53:~$ exit
logout
ubuntu@172-31-24-53:~$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/ubuntu/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ubuntu/.ssh/id_ed25519
Your public key has been saved in /home/ubuntu/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:HL_MQoYwBTrFTZuTO1oq3Qo4d5JdLz2L1R1S+y9c ubuntu@172-31-24-53
The key's randomart image is:
+--[ED25519 256]--+
|      .+..+      |
|     .+..+      |
|    +000..      |
|   +00++       |
|  .0...0.S     |
| 00 0.         |
| + 0+          |
| .000.E        |
| +00+         |
+-----+-----+
-----[SHA256]-----
ubuntu@172-31-24-53:~$ cat /home/ubuntu/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE8wAmGcQyKxSR9FipBm27DwPc2APa3eJV/yYQF ubuntu@172-31-24-53
ubuntu@172-31-24-53:~$
```

→ Add rsa_pub key of controller in ec2 user of .ssh/authorized of worker-1

```
aws  [Alt+S]  United States (N. Virginia)  Ec2
```

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE58AmGsCoyXKqR5FipRbm27TxpWCTaAPae3V/y+YQF ubuntu@ip-172-31-24-53

ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQdh5gsdwtv/p/Dvs9MEsL7UFQ/kypQ52qlDy0h6ZXKaRPCArwBGLGiI+amJu3egS0AhapThx7C2b/FTbSdfOpqgrLP0bCo9x9HFDl6xPz+47PhImagS6Zygf2K0PxOm20
Ka70bElxoQBWZlQYoSK7NDGsbriigU3jHXCMUlpf7Bqd4Iegr2lxX74LXLZgnPaZqPdANONL2tq3/yvTzBN8o2ExNh3Rhj9dWOFSISrvexlaB7SL8KlijM39ijjD3SW7tttBGywCSmVhkcEcOlkl0FHImtNrPY42fiO4xqt
zm4K5BlJq2oHSoulvkJsGPIrgxltQchewTe+AwwPKv2hVR arj

-- INSERT --                                     2,1                                     All

i-031d1cfec380f0060 (worker-1)
PublicIPs: 100.31.3.66   PrivateIPs: 172.31.31.214
```

→ Add rsa_pub key of controller in ec2 user of .ssh/authorized of worker-2

—>ansible all -m ping

```
WARNING: Invalid characters were found in group names but not replaced, use -vvvv to see details
172.31.31.214 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
172.31.20.77 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
ubuntu@ip-172-31-24-53:~$
```

→ install apache in both workers via playbook, first create a playbook and then create file with .yaml extension and add this below content

```
ubuntu@ip-172-31-24-53: ~/ansible
--
- Hosts: all
  Tasks:
    - name: Installing apache
      apt: name=apache2 state=latest
```

→ Then check the syntax using code “ansible-playbook apache.yaml --syntax-check”

→ For dry run use command “ansible-playbook apache.yaml --check”

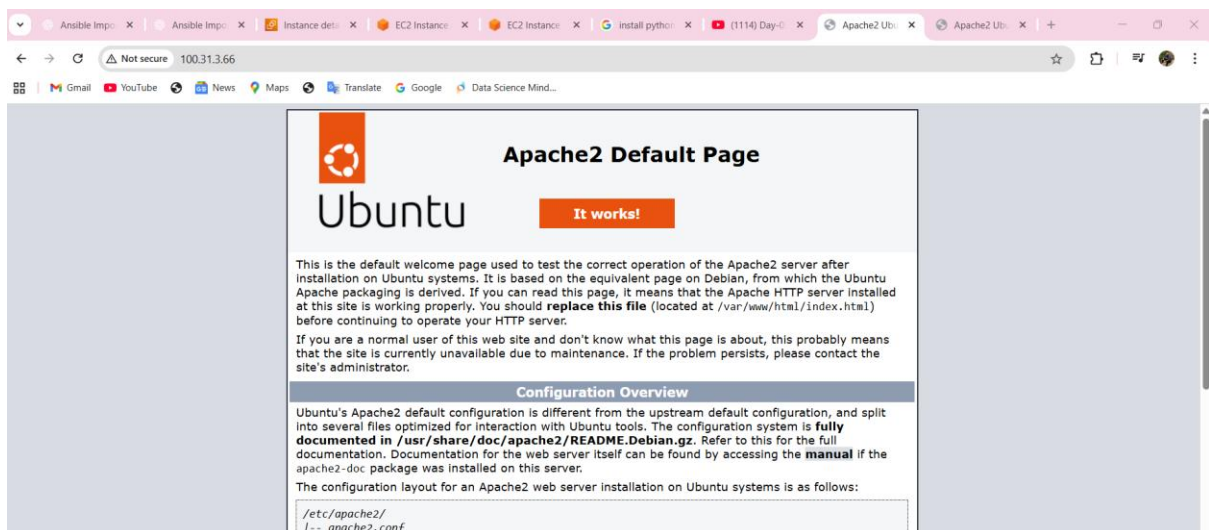
→ Use command to run “ansible-playbook apache.yaml”

```
ubuntu@ip-172-31-24-53:~/playbook$ cat apache.yaml
--
- Hosts: all
  Tasks:
    - name: Installing apache
      apt: name=apache2 state=latest
ubuntu@ip-172-31-24-53:~/playbook$ ansible-playbook apache.yaml --syntax-check
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details
[ERROR]: 'Hosts' is not a valid attribute for a Play
The error appears to be in '/home/ubuntu/playbook/apache.yaml': line 2, column 3, but may
be elsewhere in the file depending on the exact syntax problem.
The offending line appears to be:

--
- Hosts: all
  ^ here
ubuntu@ip-172-31-24-53:~/playbook$ vi apache.yaml
ubuntu@ip-172-31-24-53:~/playbook$ ansible-playbook apache.yaml --syntax-check
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details
playbook: apache.yaml
ubuntu@ip-172-31-24-53:~/playbook$ ansible-playbook apache.yaml --check
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details
PLAY [all] *****
TASK [Gathering Facts] *****
ok: [172.31.20.77]
ok: [172.31.31.214]
TASK [Installing apache] *****
ok: [172.31.20.77]
ok: [172.31.31.214]
PLAY RECAP *****
172.31.20.77      : ok=2  changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.31.31.214   : ok=2  changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu@ip-172-31-24-53:~/playbook$
```

```
ubuntu@ip-172-31-24-53: ~/playbook
--
- hosts: all
  become: yes
  tasks:
    - name: Installing apache
      apt: name=apache2 state=latest
```

→ Apache is installed in worker-1



Apache2 Default Page

Ubuntu

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

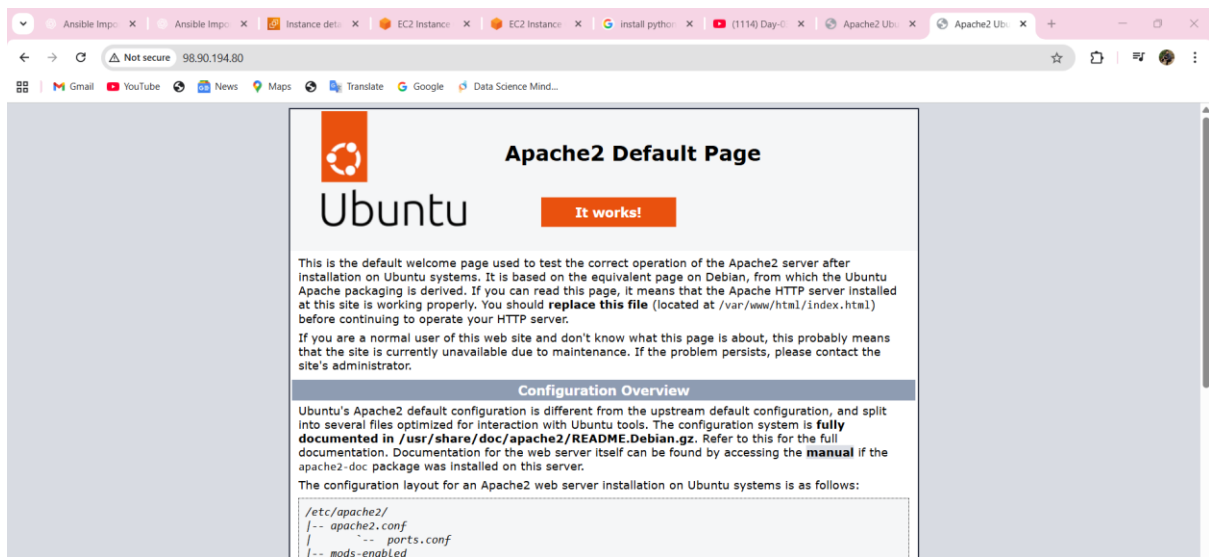
Configuration Overview

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is **fully documented in `/usr/share/doc/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

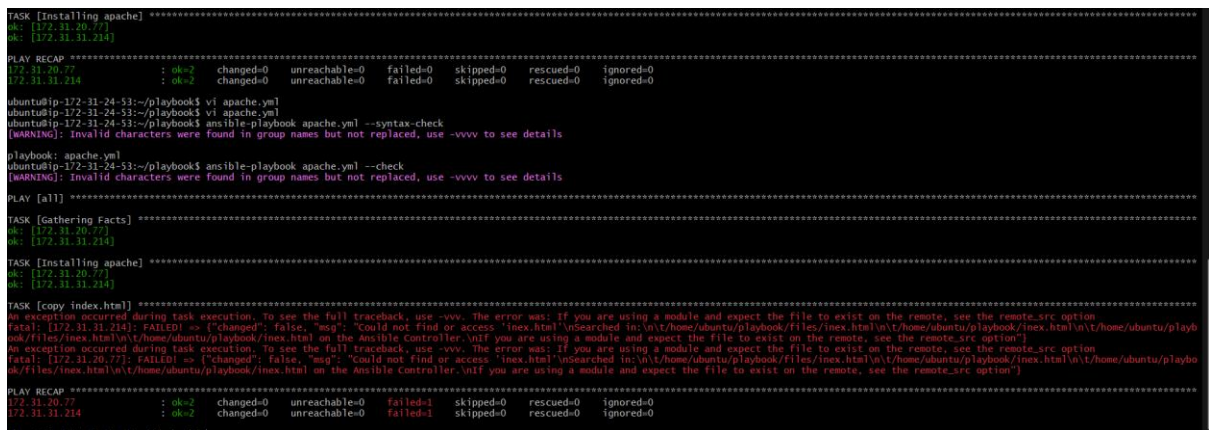
The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

```
/etc/apache2/
|-- apache2.conf
```


→ Apache is installed in worker-2

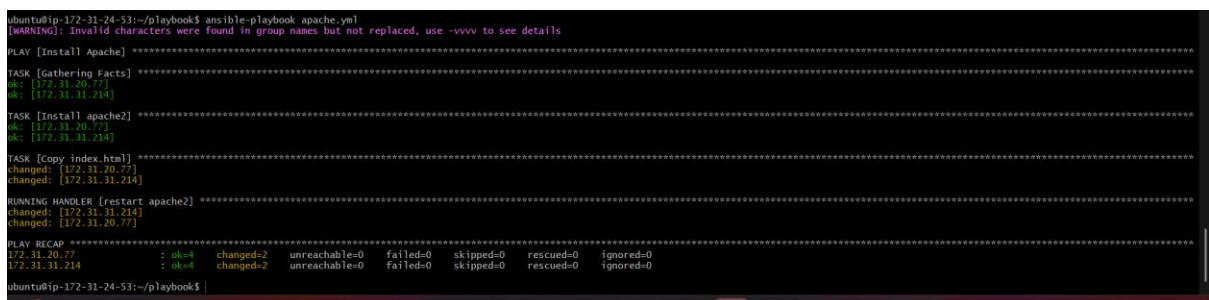


→ for copy use this command below

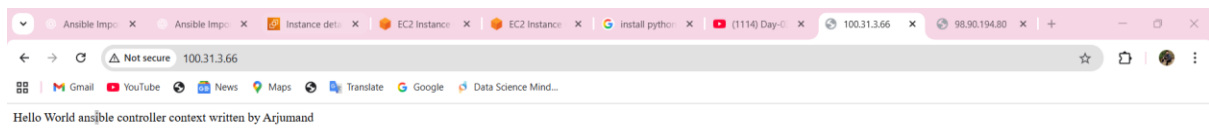


→ similarly in worker2 also

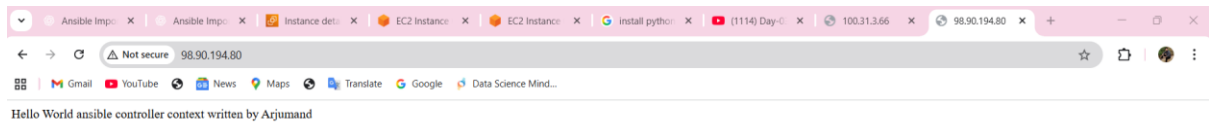
now check in ansible master server



→After copying the file index.html works on new index file in worker-1



→After copying the file index.html works on new index file in worker-2



```
ubuntu@ip-172-31-24-53:~/playbook$ ansible-playbook tomcat.yml
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details

PLAY [Install Apache Tomcat] *****

TASK [Gathering Facts] *****
ok: [172.31.31.214]
ok: [172.31.20.77]

TASK [Install Java] *****
ok: [172.31.20.77]
ok: [172.31.31.214]

TASK [Create tomcat user] *****
changed: [172.31.20.77]
changed: [172.31.31.214]

TASK [Create tomcat directory] *****
changed: [172.31.20.77]
changed: [172.31.31.214]

TASK [Download Tomcat] *****
ok: [172.31.20.77]
ok: [172.31.31.214]

TASK [Extract Tomcat] *****
changed: [172.31.20.77]
changed: [172.31.31.214]

TASK [Change ownership of Tomcat directory] *****
changed: [172.31.31.214]
changed: [172.31.20.77]

TASK [Start Tomcat] *****
changed: [172.31.31.214]
changed: [172.31.20.77]

PLAY RECAP *****
172.31.20.77      : ok=8  changed=5  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
172.31.31.214    : ok=8  changed=5  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

ubuntu@ip-172-31-24-53:~/playbook$
```

3. Write a ansible playbook to install apache tomcat.

→for installing tomcat use this command

- name: Install Apache Tomcat

hosts: all

become: yes

vars:

tomcat_version: 9.0.115

tomcat_home: /opt/tomcat

tomcat_user: tomcat

tasks:

- name: Install Java

apt:

name: default-jdk

state: present

update_cache: yes

- name: Create tomcat user

user:

name: "{{ tomcat_user }}"

shell: /bin/false

- name: Create tomcat directory

file:

path: "{{ tomcat_home }}"

state: directory

owner: root

group: root

mode: '0755'

- name: Download Tomcat

get_url:

url: "https://dlcdn.apache.org/tomcat/tomcat-9/v{{ tomcat_version }}/bin/apache-tomcat-{{ tomcat_version }}.tar.gz"

dest: /tmp/apache-tomcat.tar.gz

- name: Extract Tomcat

unarchive:

src: /tmp/apache-tomcat.tar.gz

dest: "{{ tomcat_home }}"

remote_src: yes

extra_opts:

- --strip-components=1

- name: Change ownership of Tomcat directory

file:

path: "{{ tomcat_home }}"

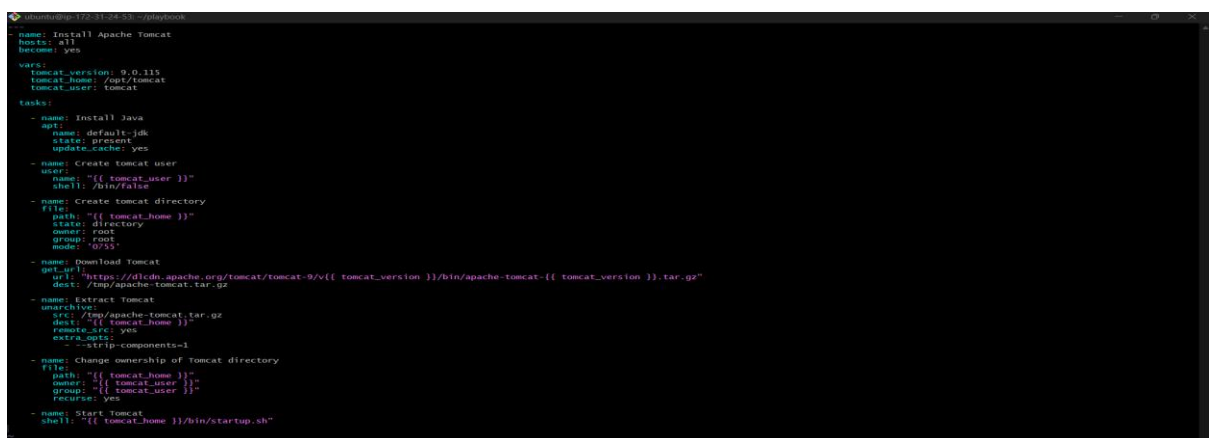
owner: "{{ tomcat_user }}"

group: "{{ tomcat_user }}"

recurse: yes

- name: Start Tomcat

shell: "{{ tomcat_home }}/bin/startup.sh"



```
ubuntu@ip-172-31-28-58: ~$ cat playbook
- name: Install Apache Tomcat
  hosts: all
  become: yes

  vars:
    tomcat_version: 9.0.115
    tomcat_home: /opt/tomcat
    tomcat_user: tomcat

  tasks:
    - name: Install Java
      apt:
        name: default-jdk
        state: present
        update_cache: yes

    - name: Create tomcat user
      user:
        name: "{{ tomcat_user }}"
        shell: /bin/false

    - name: Create tomcat directory
      file:
        path: "{{ tomcat_home }}"
        state: directory
        owner: root
        group: root
        mode: '0755'

    - name: Download Tomcat
      get_url:
        url: "https://dlcdn.apache.org/tomcat/tomcat-9/v{{ tomcat_version }}/bin/apache-tomcat-{{ tomcat_version }}.tar.gz"
        dest: /tmp/apache-tomcat.tar.gz

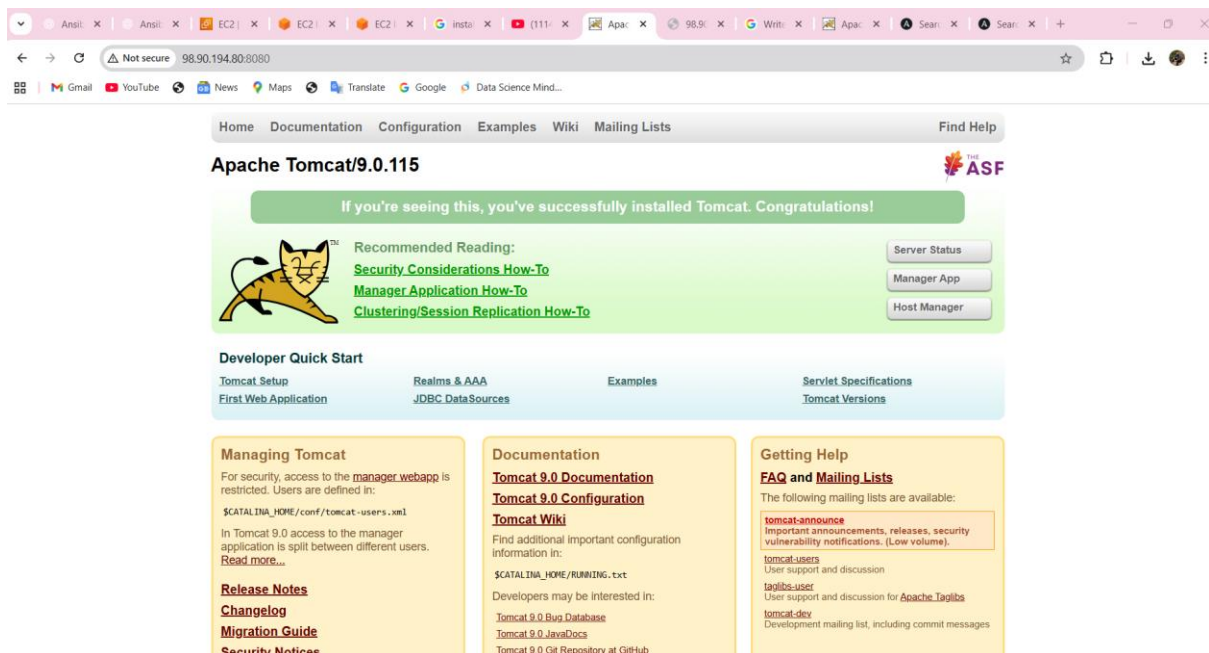
    - name: Extract Tomcat
      unarchive:
        src: /tmp/apache-tomcat.tar.gz
        dest: "{{ tomcat_home }}"
        remote_src: yes
        extra_opts:
          - --strip-components=1

    - name: Change ownership of Tomcat directory
      file:
        path: "{{ tomcat_home }}"
        owner: "{{ tomcat_user }}"
        group: "{{ tomcat_user }}"
        recurse: yes

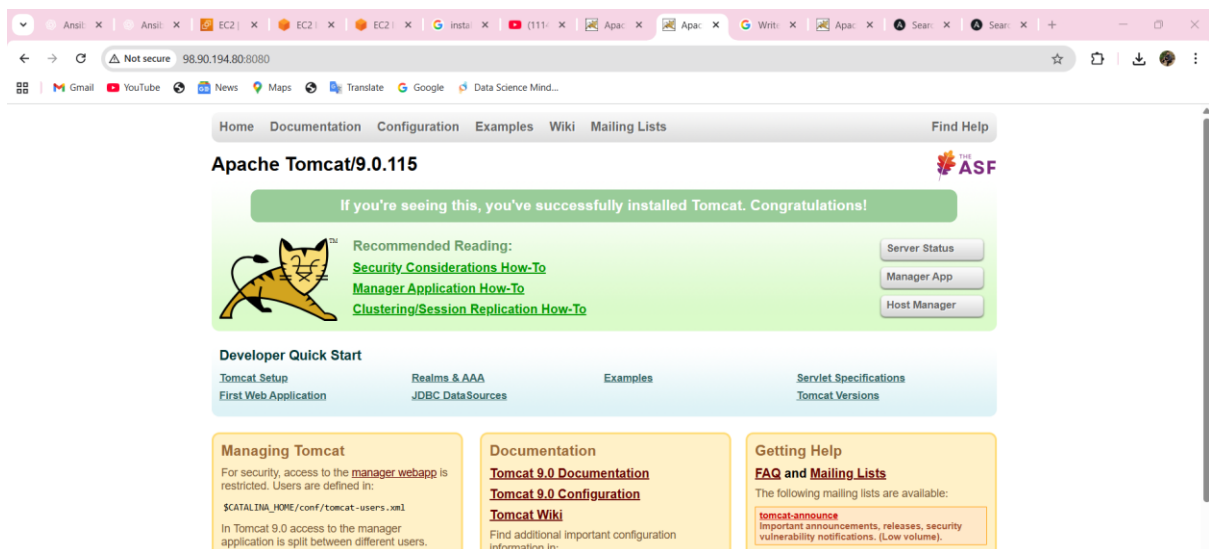
    - name: Start Tomcat
      shell: "{{ tomcat_home }}/bin/startup.sh"
```

→ Run the following command “ansible-playbook tomcat.yml”

→tomcat is installed in worker-1



→tomcat is installed in worker-2



4. Write a ansible playbook to provision one ec2 on aws.

Install pip3

sudo apt update

sudo apt install python3-pip -y

✓ Required for installing Python libraries

2 Install boto libraries (for Ansible)

pip3 install boto boto3 botocore

- ✓ Ansible uses boto3 to talk to AWS
- ✓ AWS CLI is NOT used by Ansible internally

3 Install AWS CLI (only for credentials)

curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip

unzip awscliv2.zip

sudo ./aws/install

4 Configure AWS credentials

aws configure

Provide:

- Access key
- Secret key
- Region
- Output format

```
ubuntu@ip-172-31-04-53: ~$ ansible-playbook
...
name: Provision EC2 instance
hosts: localhost
gather_facts: false

tasks:
  - name: Launch one EC2 instance
    amazon.aws.ec2_instance:
      name: "MyAnsibleInstance"
      instance_type: t2.micro
      image_id: ami-0512be01726a3de55 # Replace with your actual AMI ID
      region: us-east-1 # Replace with your desired region
      key_name: arj # Replace with your EC2 key pair name
      security_group: default # or a specific security group name/ID
      network:
        assign_public_ip: true
      tags:
        Environment: Development
        Project: AnsibleProvisioning
        register: ec2_instance_info
  - name: Display instance information
    ansible.builtin.debug:
      var: ec2_instance_info.instances[0].public_ip_address
...

```

→ Instance is launched as we can see

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availal
☐	Master_jenkin	i-04e9dd922f331d1cf	Stopped	t2.large	-	View alarms +	us-east
☐	MyAnsibleInstance	i-005bfe901b6d46efa	Running	t2.micro	Initializing	View alarms +	us-east

5. Write a ansible playbook to copy one file from node-1 to node-2.

→copy the file we can use this command

```
ubuntu@ip-172-31-24-53: ~/playbook
- hosts: worker-1
  tasks:
    - copy:
      src: file.txt
      dest: /home/ubuntu
```

→then run the command

```
ubuntu@ip-172-31-24-53:~/playbook$ vi copy.yml
ubuntu@ip-172-31-24-53:~/playbook$ ansible-playbook copy.yml
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details

PLAY [worker-1] *************************************************************************************************************************************
TASK [Gathering Facts] ************************************************************************************
ok: [172.31.31.214]

TASK [copy] *********************************************************************************************
changed: [172.31.31.214]

PLAY RECAP *********************************************************************************************
172.31.31.214      : ok=2  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

ubuntu@ip-172-31-24-53:~/playbook$
```

→copy the file from worker-1 to worker-2 we also do

```
PLAY [Fetch file1.txt from worker1] *********************************************************************
TASK [Gathering Facts] **********************************************************************************
ok: [172.31.31.214]

TASK [Fetch file from worker1 to control node] *********************************************************
changed: [172.31.31.214]

PLAY [Copy file1.txt to worker2] *********************************************************************
TASK [Gathering Facts] **********************************************************************************
ok: [172.31.20.77]

TASK [Copy file from control node to worker2] *********************************************************
changed: [172.31.20.77]

PLAY RECAP *********************************************************************************************
172.31.20.77      : ok=2  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
172.31.31.214     : ok=2  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

ubuntu@ip-172-31-24-53:~/playbook$
```

```
ubuntu@ip-172-31-31-214:~$ ls
file.txt  file1.txt
ubuntu@ip-172-31-31-214:~$ pwd
/home/ubuntu
ubuntu@ip-172-31-31-214:~$
```

i-031d1cfec380f0060 (worker-1)
PublicIPs: 100.31.3.66 PrivateIPs: 172.31.31.214

See <https://ubuntu.com/eas> or run: `sudo pro status`

```
Last login: Thu Jan 29 12:09:43 2026 from 172.31.24.53
ubuntu@ip-172-31-20-77:~$ sudo su -
root@ip-172-31-20-77:~# ls
azj  file1.txt  snap
root@ip-172-31-20-77:~#
```

i-09f70fd93f2338ed4 (worker-2)
PublicIPs: 98.90.194.80 PrivateIPs: 172.31.20.77

6. Write a ansible playbook to create different files with different names using single playbook.

→Create a file in playbook name multiplefiles.yml

→add the above code in multiplefiles.yml

→ save

```
ubuntu@ip-172-31-24-53: ~/playbook
---
- name: Create multiple files using single playbook
  hosts: all
  become: yes

  tasks:
    - name: Create different files
      file:
        path: "/tmp/{{ item }}"
        state: touch
        mode: '0644'
      loop:
        - file1.txt
        - file2.txt
        - file3.log
        - app.conf
```

—> now check the syntax-check

—> ansible-playbook multiplefiles.yml —syntax-check

When the check is perfect

Run

—> ansible-playbook multiplefiles.yml

—> now check any workers server root/

```
ubuntu@ip-172-31-24-53:~/playbook$ vi create-multiple-files.yml
ubuntu@ip-172-31-24-53:~/playbook$ ansible-playbook create-multiple-files.yml
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details

PLAY [Create multiple files using single playbook] *****

TASK [Gathering Facts] *****
ok: [172.31.20.77]
ok: [172.31.31.214]

TASK [Create different files] *****
changed: [172.31.20.77] => (item=file1.txt)
changed: [172.31.31.214] => (item=file1.txt)
changed: [172.31.20.77] => (item=file2.txt)
changed: [172.31.31.214] => (item=file2.txt)
changed: [172.31.20.77] => (item=file3.log)
changed: [172.31.31.214] => (item=file3.log)
changed: [172.31.20.77] => (item=app.conf)
changed: [172.31.31.214] => (item=app.conf)

PLAY RECAP *****
172.31.20.77      : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.31.31.214    : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

ubuntu@ip-172-31-24-53:~/playbook$
```